

Implementation Sprint I Check-in

AI-use disclosure. Drafted with Codex (GPT-5, Codex desktop app) from the W5 Canvas assignment screenshot and the approved W4 design package.

AI-drafted, student-revised. Key human-authored decisions carried into this document: preserve Docker-first scope, use a local subprocess backend only as a development smoke-test shim, keep gVisor as the W7 fallback decision point, and prioritize runnable baseline evidence over additional documentation.

Full audit trail: `docs/ai-use-log.md`.

Student: Zixuan Liang

Course: CISC 699-50-A-2026/Summer - Applied Project in Computer Information Science

Project: SafeExec - Hardened Python Execution Sandbox for LLM Agent Tool-Use

Advisor: Prof. Khalid Lateef

Instructor: Dr. Majid Shaalan

Submission date: 2026-06-09

1. Sprint I Outcome

Implementation Sprint I moved the project from design-only artifacts into a runnable engineering baseline. The repository now contains a small safeexec Python package, a structured execution result schema, a development execution backend, a Docker/gVisor command builder, a JSON POST /execute API shell, smoke-test scripts, and unit tests that verify the baseline behavior.

The committed baseline intentionally separates two concerns:

- **Development smoke path:** local backend, which runs a Python subprocess with timeout and structured output. This is not a security boundary; it exists so the service contract and tests run reproducibly on the authoring machine.
- **Target isolation path:** Docker/gVisor backend command construction, which encodes the W4 controls: no network, non-root user, read-only filesystem, dropped capabilities, no-new-privileges, PID limit, memory limit, CPU limit, and tmpfs workspace. Full target-host execution is the W6 hardening focus.

2. Repository Structure and Branch Status

The repository remains on the main branch with a clean W1-W4 history and a new W5 implementation surface:

Area	W5 baseline status
src/safeexec/	Package skeleton, models, service layer, CLI, API server, local backend, Docker/gVisor backend.
tests/functional/	Unit tests for local execution, API response shape, and Docker/gVisor command controls.
tests/adversarial/	Category taxonomy seed; program-level adversarial tests remain W6-W8 work.
benchmarks/	Local smoke benchmark seed for repeated latency timing.
deploy/	Compose wrapper and deployment notes for the API shell.
docs/05-implementation-sprint-i/	This check-in package and smoke/test evidence.

No production secrets, API keys, personal data, or regulated data are included.

3. Setup and Run Instructions

The W5 baseline requires Python 3.11 or newer and no third-party dependencies.

```
make smoke
make test
make api
```

Manual CLI example:

```
PYTHONPATH=src python3 -m safeexec --backend local --code "print('hello')"
```

Manual API example after make api:

```
curl -s http://127.0.0.1:8080/execute \
-H 'Content-Type: application/json' \
-d '{"code": "print(2 + 2)"}'
```

4. Smoke-Test Evidence

make smoke executed successfully on 2026-06-09. The captured output is stored in docs/05-implementation-sprint-i/smoke-output.txt. It demonstrates that the service layer accepts a Python program, runs it through the local backend, and returns structured JSON with status, backend, exit_code, stdout, stderr, duration_ms, contained, containment_reason, and limits metadata. make test also executed successfully. The captured output is stored in

docs/05-implementation-sprint-i/test-output.txt. Current coverage is narrow by design but aligned with W5: normal local execution, timeout handling, API response shape, and Docker/gVisor command control construction.

5. Risk and Issue Log

ID	Risk / issue	Status	Mitigation / next action
W5-R1	Local subprocess backend is not a security sandbox.	Known limitation	Label it explicitly as dev-only; do not use it for containment claims.
W5-R2	Docker/gVisor execution not yet verified from this local workspace.	Open	Run the Docker backend on the Ubuntu droplet in W6 and capture output.
W5-R3	Functional corpus is still a seed, not the planned HumanEval/MBPP subset.	Open	Add curated subset manifest and first >=30 functional programs in W6.
W5-R4	Adversarial suite contains taxonomy only.	Open	Author program-level tests by category after Docker hardening is exercised.
W5-R5	API shell uses Python stdlib HTTP server rather than FastAPI.	Accepted W5 tradeoff	Preserve response contract now; migrate only if FastAPI adds value after backend path is stable.

6. Sprint Reflection and Forward Plan

The most useful outcome of Sprint I is that SafeExec now has a runnable spine: request model, backend interface, execution result, API endpoint, CLI, and tests all agree on the same contract. This reduces the W6 risk because Docker hardening can be added behind the existing interface rather than designed at the same time as the API and test harness.

The next sprint should stay implementation-first:

1. Execute the Docker backend on the Ubuntu droplet and capture the first Docker POST /execute evidence.
2. Convert the Docker command controls into a documented hardening profile.
3. Add the first functional corpus manifest with HumanEval/MBPP task IDs and student-authored smoke programs.
4. Add initial adversarial programs only after the Docker boundary is running,

so each probe has a real expected contained outcome.

5. Update README and the engineering log with exact command outputs from the target host.

7. Canvas Submission Contents

Recommended Canvas submission:

- `Implementation-Sprint-I-Check-in.pdf`
- `Implementation-Sprint-I-Check-in.docx`
- Repository link or commit hash for the W5 baseline
- Optional evidence attachments: `smoke-output.txt`, `test-output.txt`, and `benchmark-smoke-output.txt`